

CSCE 611 OPERATING SYSTEMS · MODULE 11 · LECTURE NOTES

Protection and Isolation

Containers, virtual machines, and what each actually isolates.

MODULE OUTCOMES

- Explain users, groups, and the limits of discretionary access control.
- Explain capabilities and the principle of least privilege.
- Explain namespaces and cgroups, and what a container really is.
- Compare container and VM isolation honestly.
- Explain the kernel attack surface and why it matters.

1 Classical Unix protection

Every process carries a user ID and group IDs; every file carries an owner, a group, and nine permission bits. Access is decided by comparing them. This is **discretionary access control**: the owner of a resource decides who may use it.

It is admirably simple and it has aged poorly in three specific ways.

- **root bypasses everything.** A single account with unlimited authority means any compromise of a root process is a total compromise.
- **Granularity is wrong.** Permissions attach to files, so expressing 'this program may read these three files and open outbound connections to this host' requires contortions.
- **Ambient authority.** A process automatically holds *all* of its user's privileges, whether it needs them or not. A text editor can delete your entire home directory because you can. A bug in any program is a bug with your full authority behind it.

SETUID, AND WHY IT IS A PERSISTENT WEAKNESS

Some operations genuinely require privilege but must be available to ordinary users: `passwd` must modify `/etc/shadow`, `ping` historically needed raw sockets.

The **setuid** bit causes a program to run with the privileges of the file's owner rather than the invoking user. `passwd` is `setuid` root.

The consequence is that every `setuid`-root binary on the system is a potential privilege escalation: find an exploitable bug in any of them and you obtain root. This has historically been one of the most productive categories of local vulnerability.

The principled response is **least privilege**: grant the specific power required rather than all powers. That is what capabilities provide, and why `ping` now uses `CAP_NET_RAW` instead of being `setuid`.

2 Narrowing privilege

Mechanism	What it does	Example
Capabilities	Divide root's authority into roughly forty distinct powers that can be granted individually.	A web server receives <code>CAP_NET_BIND_SERVICE</code> to bind port 80 and nothing else — it cannot read arbitrary files or load kernel modules.
seccomp	Restrict which system calls a process may issue, with a BPF filter.	A sandboxed renderer permits around twenty calls and blocks the rest. Chrome and OpenSSH use this heavily.
chroot / pivot_root	Change the process's apparent filesystem root.	Weak alone — a privileged process can escape — but a useful building block when combined with the others.
Mandatory access control (SELinux, AppArmor)	Policy set by the administrator and enforced regardless of the file owner's wishes.	Even root is constrained by policy. Changes the model from discretionary to mandatory.
Privilege dropping	Start with the privilege needed for initialisation, then permanently relinquish it.	Bind to port 80 as root, then <code>setuid</code> to an unprivileged account before handling any request.

THE SYSCALL SURFACE IS THE THING TO SHRINK

A container's isolation is enforced *by the kernel*. The kernel's attack surface is therefore its system call interface — around 400 calls, several of them decades old, complex, and rarely exercised.

A kernel bug reachable through any of them is a container escape: the attacker ceases to be confined.

A typical application needs perhaps forty system calls. A `seccomp` profile blocking the other 360 eliminates most of the reachable attack surface for the cost of a configuration file.

This is why serious container runtimes apply a default `seccomp` profile, and why 'which system calls does this workload actually need' is a productive security question rather than a pedantic one.

3 What a container actually is

THERE IS NO CONTAINER OBJECT IN THE KERNEL

A container is not a kernel abstraction. It is an ordinary process with several independent kernel features applied to it, assembled by a runtime such as runc into something that behaves like an isolated machine.

Understanding this clarifies both the capabilities and the limits: whatever the features isolate is isolated, and whatever they do not, is not.

Feature	Provides	Namespace types
Namespaces	A private view of an otherwise global resource.	PID (its own process tree, where it is PID 1); mount (its own filesystem view); network (its own interfaces and routing table); user (UID mapping, so container root is an unprivileged host user); UTS (hostname); IPC ; cgroup ; time .
cgroups	Resource limits and accounting.	CPU shares and quotas, memory limits, block I/O bandwidth, process count. This is what stops one container starving the others.
seccomp	System call restriction.	Shrinks the kernel attack surface, as above.
Capabilities	Privilege restriction.	A container's 'root' holds a reduced capability set.

3.1 Containers against virtual machines

	Container	Virtual machine
Kernel	Shares the host's.	Runs its own.
Isolation enforced by	Kernel features: namespaces, cgroups, seccomp.	Hypervisor plus hardware virtualisation support.
Start-up	Milliseconds.	Seconds.
Overhead	Essentially none.	A full kernel's worth of memory, plus some trap overhead.
Attack surface	The entire system call interface — large, old, complex.	The hypervisor interface — far smaller and more heavily audited.
Guest OS	Must match the host kernel.	Any.

CONTAINERS ARE A WEAKER BOUNDARY, AND USUALLY THE RIGHT CHOICE

Stating this plainly is more useful than the marketing framing. A container escape requires one exploitable kernel bug reachable from a permitted system call, and such bugs are found regularly.

Containers isolate workloads you already trust from one another — your own services, built by your own team, from your own code. The density and startup advantages are enormous and the residual risk is acceptable.

VMs isolate mutually distrusting parties — different customers sharing hardware. The smaller, hardware-assisted boundary is worth the overhead.

The middle ground is **lightweight VMs**: a minimal guest kernel and a stripped hypervisor, giving VM-grade isolation with container-like startup times. AWS Firecracker and Kata Containers are the examples, and this is what serverless platforms actually run — precisely because they execute untrusted customer code.

4 Virtualisation mechanics

A guest kernel is written on the assumption that it runs in supervisor mode with full control of the hardware. It does not.

Mechanism	How it works
Hardware virtualisation (Intel VT-x, AMD-V, ARM EL2)	Adds a privilege level <i>beneath</i> the kernel's. The guest runs in a mode that looks like supervisor mode; privileged operations trap to the hypervisor, which emulates them and resumes the guest. Before this existed, the same effect required binary translation or paravirtualisation.
Nested paging (EPT, NPT, Stage-2)	Two levels of translation in hardware: guest-virtual to guest-physical (by the guest's page tables) and guest-physical to host-physical (by the hypervisor's). Before this, hypervisors maintained <i>shadow page tables</i> and trapped every guest page table modification — ruinously expensive. Nested paging is the single development that made virtualisation cheap enough to be ubiquitous.
Paravirtualisation (virtio)	The guest knows it is virtualised and uses efficient ring-buffer interfaces for disk and network rather than driving emulated hardware registers that would trap on every access.
Device passthrough (with an IOMMU)	Assign a physical device directly to a guest. The IOMMU translates and confines the device's DMA (Module 10), making this safe.

5 The isolation spectrum

Mechanism	Boundary enforced by	Strength
Process	Page tables.	Weak. Shares the kernel; full system call access; shares all microarchitectural state.
Container	Namespaces, cgroups, seccomp, capabilities.	Moderate. Good against accident and against unprivileged attackers; one kernel bug from failure.
Lightweight VM	Hypervisor with a minimal guest.	Strong , with fast startup. The current best answer for untrusted code at density.
Full VM	Hypervisor plus a complete guest kernel.	Strong . The standard for multi-tenant infrastructure.
Separate hardware	Physics.	Strongest available, and still not absolute.

THERE IS NO PERFECT ISOLATION

Even separate machines share a network, a power supply, and a physical environment.

Within a machine, **side channels** cross every boundary described here. Cache timing, branch predictor state, and speculative execution leak information between processes, between containers, and between virtual machines — Spectre and its relatives (CSCE 614 Module 04) were effective across all of them.

Mitigations exist and cost real performance. The honest summary is that isolation is a matter of degree and of cost, not a binary property, and that choosing a mechanism means choosing an acceptable residual risk rather than eliminating risk.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
OSTEP — security chapters	Access control and the authentication model.
Linux kernel documentation — namespaces, cgroups v2, seccomp	The primary reference for what a container is actually built from.
Agache et al. — Firecracker: Lightweight Virtualization for Serverless Applications (free)	Why serverless platforms use micro-VMs rather than containers, argued with data.
Saltzer & Schroeder — The Protection of Information in Computer Systems (1975, free)	The paper that stated least privilege and the other design principles. Still the clearest statement of them.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write a program that drops privileges correctly: bind a privileged port as root, then permanently relinquish root. Verify with `/proc/self/status` that the capability set is empty.
- 2 Write a seccomp filter permitting only the system calls your program needs. Determine the set empirically with `strace`, then verify that a blocked call terminates the process.
- 3 Build a minimal container by hand: `clone()` with new PID, mount, and network namespaces, then `pivot_root`. Confirm the child sees itself as PID 1.
- 4 Add a cgroup memory limit to your hand-built container and demonstrate that exceeding it triggers the OOM killer inside the container only.
- 5 Compare startup time and memory footprint for the same trivial workload as a process, a container, and a VM.
- 6 Enumerate the `setuid-root` binaries on a Linux system. Pick one and identify which capability it would need if rewritten to use capabilities instead.
- 7 Read a default Docker seccomp profile and count how many of the roughly 400 system calls it permits.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is ambient authority, and why is it a problem?
- 2 Why is every `setuid-root` binary a potential privilege escalation, and what is the principled alternative?
- 3 Why is reducing the system call surface the most effective single hardening measure for a container?
- 4 What is a container made of? Name four kernel features and what each provides.
- 5 Why is a container a weaker isolation boundary than a VM, and when is that acceptable?
- 6 What does nested paging replace, and why did it matter so much?
- 7 Why is there no such thing as perfect isolation?